

ISP Studio IP Generator (编组 → FPGA Verilog IP 包) 实施计划

目标

在 ISP Studio 编组右键菜单新增「生成Verilog IP」入口：把编组（像素流式子图）导出为可综合 Verilog IP 包，针对 FPGA 行级实时处理优化，实时性 ↔ RTL 规模的权衡**暴露为 IP 定制参数**（Verilog `parameter`，Vivado 等工具的定制 GUI 可直接改）。

已确认的用户决策

项	决策
目标平台	下拉可选 6 项：① AMD/Xilinx Vivado ② Intel Quartus ③ Microchip Libero ④ Lattice Radiant/Diamond ⑤ Efinix Efinity ⑥ 通用 Verilog； v1 实现 ①③⑥ ，②④⑤ 列出但禁用并标注「待完成」
节点范围	纯整数节点集（下表白名单）
视频输入	GUI 可配置物理层接口：DVP 并口 / LVDS (2/4/8 lanes) / MIPI CSI-2 (1/2/4/8 lanes) / 直通
权衡旋钮	暴露为 IP 定制参数（generate-if 分档实现）
验证	testbench + 一键仿真（探测 iverilog，有一键跑 PASS/FAIL，没有只出文件）

总体设计

复用既有 IR (核心思路，不另起炉灶)

黑盒行级流水的三层架构中，**校验层与规划层直接复用，只新写 Verilog 发射层**：

- 校验：新建 `validateGroupIpExport`，口径 = `validateGroupBlackBoxExport`（全帧统计/跨帧/数据环在 FPGA 上同样需要帧缓冲，拒绝集天然成立）+ IP 白名单收窄 + 参数检查。

- 规划: `planGroupC` (group_c_plan.dart:165) + `planGroupStream` (stream_plan.dart:305) **原样复用**。映射关系:
 - `StreamLineBuffer` (rows=2r+1, 槽 t%rows) → BRAM 行缓冲组 (推断式 RAM + synthesis attribute) ;
 - `StreamStage.chain` 融合链 → 同一使能域流水级 (中间值不物化) ;
 - `streamDelays` /延迟均衡 FIFO → reconvergent 路径延迟匹配 (规划层已算出深度) ;
 - 分离趟 (morphology 水平+垂直) → 经典可分离滤波器结构;
 - 尾部冲刷 → 帧尾 drain (sof/eol 协议自然表达) ;
 - `maxDelay` → 总延迟报告。
- 目标体系: IP 的「FPGA 厂商」目标 (6 家) 与 C 代码导出的「CPU 目标」(`GroupCTarget`, `codegen/group_c_target.dart`, 6 个) 是**两套并列目标体系**, 互不沿用。厂商元数据 (显示名/封装文件名/导入说明) 仿 `GroupCTargetInfo` extension (`displayName`/`archNote`/`cFlags`) 单独定义 `IpVendor` enum (v1 实现 Vivado/Libero/通用, Quartus/Lattice/Efinix 列出但禁用) , 贯通 `buildGroupIpFiles` 与对话框。

流接口 (全目标统一核心 + 厂商薄封装)

- 核心模块 `<top>_core.v`: 通用像素流 `in_valid/in_sof/in_eol/in_data` (+ 输出同形) , **全程带 ready 握手** (`advance = valid & ready`) , 这是面积档多周期折叠的前提。
- 厂商封装各自一个文件, 只转接口不含逻辑:
 - Vivado: AXI4-Stream (tvalid/tready/tdata/tuser=sof/tlast=eol) + `package_ip.tcl` ;
 - Libero: valid/ready 裸流封装 + 导入说明 (tcl 留 v2) ;
 - 通用: 核心直接即顶层。
- 节点参数**烘焙进生成代码** (与 C 导出同口径) ; 运行时寄存器/AXI-Lite 留 v2。

视频输入物理层 (GUI 可配置的 sensor 接口适配层)

ISP 核心只消费标准化像素流 (`in_valid/in_sof/in_eol/in_data`) 。sensor 侧的物理接口 (LVDS / DVP / MIPI CSI-2) 由**输入适配层**桥接: 生成器按 GUI 选择的接口类型产出对应的顶层端口 + 解串/对齐/解包逻辑, 把物理信号归一为像素流后喂给核心。接口选择是 IP 定制参数, 直接驱动顶层端口列表与适配 RTL。

接口	lanes	关键参数	适配逻辑 (生成器产出)	厂商依赖
<code>none</code> (直通)	—	—	不生成适配层, 核心像素流端口直出	无

接口	lanes	关键参数	适配逻辑 (生成器产出)	厂商依赖
			(测试/级联用)	
DVP 并口	并行 data	DVP_W (8/10/12/16)、同步风格 (HSYNC+VSYNC+DE / HREF+VSYNC)、信号极性	纯并行: pixel clock + 同步 → 像素流 (时序对齐)	无
LVDS (sub-LVDS 视频)	2/4/8	LVDS_LANES 、 LVDS_BITW 、 LVDS_ORDER (排序规则)、SDR/DDR	位/字对齐 (bit-slip) + 解串 (IDDR/ISERDES 原语) + 按排序规则还原像素	各厂商解串原语 (Xilinx ISERDESE / Intel ALTLVDS / Microchip IO)
MIPI_CSI2	1/2/4/8	CSI_LANES 、 CSI_DT (RAW8/10/12/14)、 CSI_VC (0-3)	实例化厂商 CSI-2 RX IP (D-PHY + 包解析) + RAW 解包适配器 → 像素流	各厂商 RX IP (Xilinx MIPI CSI-2 RX Subsystem / Intel MIPI CSI-2 RX II / Microchip PolarFire MIPI CSI-2)

LVDS 数据排序规则 (LVDS_ORDER**) 是 LVDS 的关键参数**——不同位宽/不同 sensor 的 lane→bit 映射不同，直接决定线速率与像素还原：

- **bit-split**：每条 lane 承载同一像素的 1 bit (4 lanes × N bit 逐位串行) ；
- **pixel-split**：每条 lane 承载完整像素 (或字节) 串行；
- 方向 MSB-first / LSB-first；DDR 时另需 **LVDS_EDGE** (偶数像素=上升沿/下降沿) 。
- **派生线速率** = 像素率 × 每像素 bit ÷ lanes ÷ (DDR 时 ÷2)，GUI 只读展示并对照目标器件 LVDS 上限做校验提示；非法组合 (如 bit-split 且 lanes≠BITW) 在 GUI 层 enablement 禁止。

MIPI CSI-2 全栈 RX (D-PHY + LP/HS + 包解析 + ECC/CRC + lane align) 不自行实现：v1 只做「厂商 RX IP 实例化 + 参数透传 + RAW 解包适配器」，把厂商 IP 输出 (多为 AXI4-Stream) 归一为像素流；D-PHY 差分对/时钟引脚按厂商 IP 约束由顶层透出。全栈自研留 v2。

验证口径：EXACT/TOLERANCE 逐位对拍仍针对 core（像素流级，testbench 直喂 golden_in.hex）；vidin 适配层另做**结构/行为仿真**（LVDS 串行激励 → 像素流还原；DVP 时序 → 像素流；CSI-2 用厂商 RX IP 自带 BFM 或留 v2）。输出侧物理接口（LVDS/MIPI TX 等）为对称扩展，v1 仅做输入，输出沿用像素流/AXI4-Stream 封装。

权衡参数（IP 定制参数，顶层 parameter + generate-if）

参数	取值	语义
PIX_BITS	默认=位深推导 (lutDomainMaxOf)	像素位宽；LUT RAM 深度随之
MAX_WIDTH	默认 4096	行缓冲深度（BRAM 用量报表）
GSTRATEGY	0 面积 / 1 平衡 / 2 性能（默认 2）	窗口算子实现分档
FOLD	1/2/4（面积档默认 2）	窗口算子折叠因子：每像素 FOLD 拍、算子复用，吞吐 1/FOLD px/clock

分档实现（仅窗口节点有两套变体；点操作/LUT 恒 ll=1，面积占比小）：

- **性能：**窗口全展开（比较树/移位网络全组合），乘法器 use_dsp 推断，ll=1；
- **平衡：**全展开但乘法器绑 LUT 实现、比较树分组打拍（更多 FF 换逻辑深度）；
- **面积：**FOLD 拍每像素，窗口 tap 串行复用同一组算子（行缓冲多读口改时分），吞吐降但 DSP/LUT 大幅省。
- 生成器同时产出**资源/延迟估算报告**（每节点 DSP/LUT/BRAM 估算 + 各档对比，写入 readme 与页面附加区），供用户针对性选档。

IP 定制 GUI 规划（参照 Vivado「Customize IP」对话框）

生成的 IP 在 Vivado 中双击打开时，经 IP Packager 的 Customization GUI (xgui tcl) 呈现为**分页 + 控件化 + 参数联动**的配置界面，而不是裸 parameter 列表。布局与控件规划：

Page 0「Video In 视频输入接口」

参数	控件	选项/范围	联动/说明
VIN_TYPE	comboBox	无/直通(none) · DVP 并口 · LVDS · MIPI CSI-2（默认 none）	驱动下方子参数联动显示/隐藏
DVP_W	comboBox	8 / 10 / 12 / 16	仅 DVP；数据位宽

参数	控件	选项/范围	联动/说明
DVP_SYNC	comboBox	HSYNC+VSYNC+DE / HREF+VSYNC	仅 DVP; 同步风格
LVDS_LANES	comboBox	2 / 4 / 8	仅 LVDS
LVDS_BITW	comboBox	8 / 10 / 12	仅 LVDS; 每像素 bit
LVDS_ORDER	comboBox	bit-split / pixel-split (× MSB/LSB)	仅 LVDS; 排序规则
LVDS_DDR	comboBox	SDR / DDR	仅 LVDS; 双沿
CSI_LANES	comboBox	1 / 2 / 4 / 8	仅 MIPI CSI-2
CSI_DT	comboBox	RAW8 / RAW10 / RAW12 / RAW14	仅 MIPI
CSI_VC	comboBox	0–3	仅 MIPI
派生线速率/像素时钟	textEdit 只读	随接口/位宽/lanes 联动	线速率 = 像素率×bit/lanes (÷DDR)，对照器件上限提示

Page 1 「General 基本配置」

参数	控件	选项/范围	联动/说明
PIX_BITS	comboBox	8 / 10 / 12 / 14 / 16 (默认=位深推导)	像素位宽; 决定数据通路宽度与 LUT RAM 深度
MAX_WIDTH	comboBox	640 / 1280 / 1920 / 2048 / 4096 / 8192	行缓冲最大行宽, 直接决定 BRAM 用量
LINE_BUFFER_COUNT 等派生项	textEdit 只读 (enablement=false)	自动计算	行缓冲条数、估算 BRAM (36K 块数)、总延迟 (行/拍) ——只读展示, 帮用户当场评估资源

参数	控件	选项/范围	联动/说明
<code>GSTRATEGY</code>	comboBox	性能优先(2) / 平衡(1) / 面积优先(0), 默认性能	窗口算子实现分档, tooltip 写明各档取舍
<code>FOLD</code>	comboBox	1 / 2 / 4	仅 <code>GSTRATEGY</code>=面积优先时可编辑 (enablement 联动) ; 其余档锁定为 1 并灰显, tooltip 注明原因
吞吐/资源摘要	textEdit 只读	随 <code>GSTRATEGY</code> / <code>FOLD</code> 联动刷新	当前档吞吐 (px/clk) 、估算 DSP/LUT/BRAM——派生 parameter

GUI 工程细节:

- 每参数配 `display_name` (英文) 与 `tooltip` (中文说明取舍与影响) ; 取值合法性在 tcl 层用 `value_pairs` / range 校验约束, 非法组合 (如 `FOLD`>1 但 `GSTRATEGY`≠面积、LVDS bit-split 但 lanes≠BITW) 经 **enablement 禁止**而非事后报错;
- 实现位置: `package_ip.tcl` 内含 `ipgui::add_page` / `ipgui::add_param` 段 (或生成独立 `xgui/<top>.tcl` 由打包脚本装配——二选一, 实现时按 Vivado 版本兼容性定) ; generate-if 在 RTL 内按 parameter **真实生效**, GUI 只是配置入口;
- Libero / 通用目标无等价 GUI: 参数仍列在顶层 parameter (Libero SmartDesign 的 generics 面板可见可改) , README 给同一张参数表。

应用内对话框与 Vivado GUI 同构: `ip_gen_dialog.dart` 的选项区按同样三组布局 (视频输入接口 / 基本配置 / 优化策略, `VIN_TYPE` 与 `FOLD` 均随上层档联动禁用) , 所选项写入生成代码的 parameter **默认值**——应用里选的是「出厂默认」, 用户在 Vivado 定制界面可随时再改。对话框附提示行: 「以上参数在 Vivado Customize IP 界面中可再次修改」。

v1 节点白名单与定点口径 (全部已有逐位可复现先例)

类别	节点	RTL 口径	对拍口径
LUT 类	multi_band_eq (lut_fixed Q14, <code>bb_clamp_q14</code>) 、gamma、levels_curves、highlight(clip)、pseudo_color	生成期烘焙表 → ROM/RAM 查表	逐字节 (multi_band_eq 要求 <code>codegenMode=lut_fixed</code>)
容差 LUT	color_controller (lut 表 double→生成期烘焙 Q14 定点表)	同 lut_fixed	逐像素 diff ≤1

类别	节点	RTL 口径	对拍口径
矩阵	ccm (Q20, isp_ccm.h:26)	int 乘加 + >>20	逐字节
CSC	csc_rgb2yuv / csc_yuv2rgb (Q16, isp_csc_common.h)	Q16 乘加 (含 limited range 整数乘加)	逐字节
窗口	morphology (纯比较树)、demosaic bilinear (移位平均; 边界小整数除法 count∈{2,3,4,6} 做精确常量除)、dpc (中位数比较网络, Bayer 半径 2 需 5 行缓冲)、highlight recover (小整数除法)	见分档	逐字节
连线/汇合	rgb/yuv/hsl splitter/combiner、mux4、blender/adder/multiplier (权重烘焙 Q16)	连线/定点乘加	逐字节

边界语义：C 侧「越界裁剪」与 RTL「边界复制」的分歧按**裁剪语义**实现（带 count 的精确小常量除法），保逐位一致；gaussian 类不在 v1。

产出物文件集（每组一个目录，top 名 `isp_ip_<组名净化>`）

isp_ip_<组>_core.v	流水线核心（像素流 in→out，阶段/行缓冲/延迟均衡，generate-i
isp_ip_<组>_vidin.v	视频输入适配层（VIN_TYPE 参数化：DVP/LVDS/MIPI → 像素流，含
isp_ip_<组>_top.v	顶层装配：vidin → core → 输出（物理端口按 VIN_TYPE 参数化）
isp_ip_<组>_axis.v	Vivado AXI4-Stream 输出封装 [目标=Vivado]
isp_ip_<组>_libero.v	Libero 输出封装 [目标=Libero]
tb_isp_ip_<组>.v	testbench（\$readmemh 输入 + FNV-1a/容差对拍，参数化）
golden_in.hex	LCG 输入帧（应用内 Dart 生成）
golden_out.hex	期望输出帧（仅容差模式需要）
golden_hash.txt	期望 FNV-1a（精确模式）
package_ip.tcl	Vivado IP 打包脚本（含 ipgui 定制界面定义） [目标=Vivado]
README.md	接口时序/参数表/资源估算/Libero 与通用导入说明

testbench 两模式由生成器按编组节点构成自动选择：纯整数节点编组 → EXACT（哈希比对）；含 color_controller → TOLERANCE（逐像素 $|diff| \leq 1$ ）。

新增/改动文件

新增（codegen，纯 Dart 风格与 group_c_export_bb 一致，手写 StringBuffer 不加依赖）

1. `lib/modules/isp_studio/codegen/ip_validate.dart` — `validateGroupIpExport(graph, group)`：白名单 + demosaic Bayer/bilinear 限制 + multi_band_eq lut_fixed 要求 + 复用 `streamPlanDataCycleError`。
2. `lib/modules/isp_studio/codegen/ip_gen_plan.dart` — `planGroupIp(...)`：在 `GroupStreamPlan` 之上补 IP 侧信息（位深 `lutDomainMaxOf`、总延迟、BRAM/DSP/LUT 估算、策略参数默认值）；同文件（或独立 `ip_target.dart`）定义 `IpVendor` enum（6 家厂商 + 通用，元数据仿 `group_c_target.dart` 的 `GroupCTargetInfo`：displayName/封装文件名/导入说明）。
3. `lib/modules/isp_studio/codegen/node_v_stream.dart` — 逐类型 Verilog 片段发射器（点操作表达式 / LUT ROM 表 / 窗口算子 unroll+fold 两变体）。对应 `node_c_stream.dart` 的角色。LUT 表烘焙复用 **node_c_stream 各节点发射器调用的同一批 Dart 侧函数**（`multiBandLuts`/`hslBandLuts`/`levelsCurveLut`/`pseudoColorLuts`/`highlightClipLut`/`rightContrastAdjustLut` 等），只换格式化器：C 侧 `cU16Table`/`cI32Table`/`cF64Table` → Verilog 侧新写 `$readmemh`/`localparam` ROM 表格式化。
4. `lib/modules/isp_studio/codegen/vidin_stream.dart` — 视频输入适配层发射器（DVP/LVDS/MIPI 参数化 RTL + 厂商原语例化 + LVDS 排序规则还原）。与 `node_v_stream` 同层、输入侧专用；`VIN_TYPE` 决定顶层端口与适配逻辑分支。
5. `lib/modules/isp_studio/codegen/group_ip_export.dart` — `buildGroupIpFiles(graph, group, options) → Map<String,String>`：vidin/核心/封装/tb/golden/tcl/README 组装 + `exportGroupIpPackage(..., dir)` 写盘。对应 `group_c_export_bb.dart` 的角色。
6. `lib/modules/isp_studio/codegen/iverilog_sim.dart` — `detectIverilog()`（PATH + 常见安装根，仿 `detectArmGcc`）+ `runIpSimulation(files, {onOutput})`：写 systemTemp 唯一目录 → `iverilog -g2012` → `vvp`，流式输出（仿 `c_compile.dart` 的 `runStep`），解析 PASS/FAIL，全程超时保护。

新增 (widgets)

7. `lib/modules/isp_studio/widgets/ip_gen_dialog.dart` — 生成选项对话框：目标下拉（6 项，Quartus/Lattice/Efinix 禁用注「待完成」）、**选项区与 Vivado Customize IP 界面同构**（「视频输入接口」组：VIN_TYPE + 子参数联动；「基本配置」组：位深/行宽；「优化策

略」组：GSTRATEGY 单选 + FOLD 联动禁用，附「参数在 Vivado 定制界面可再改」提示行）、iverilog 探测状态行。静态部分仿 `showIspGroupNamingDialog`，联动与探测状态仿 `group_compile_dialog`。

8. `lib/modules/isp_studio/widgets/ip_generator_page.dart` — IP 标签页：页头（仿 `group_code_page` `buildHeader`）+ FutureBuilder（留 `filesBuilder` 注入点供测试）+ CFileList（分组：视频输入/核心/接口封装/仿真/脚本文档）+ CodeArea（高亮 `'v'`，`syntax_highlighter` 已有 `_verilog` 模式）+ 工具栏「生成选项 / 导出IP包 / 一键仿真」（仿真输出复用 `CodeCompileArea` 的终端面板模式；工具栏结构仿 `code_browser.dart:1204` 自组，不复用写死编译按钮的 `CodeCompileArea`）。

改动（挂载点，行号已核实）

9. `lib/providers/isp_studio_state.dart` — :274 后（`openGroupBlackBoxCodeTab` 之后）加 `openGroupIpTab(String groupId, {IpVendor vendor})`（key `'gip:<id>@<vendor>'`，与 `'group:<id>@<target>'` 同口径，同一编组可同开多家厂商 IP 标签页）；`ungroup`（:1543）联动关闭把 `'gip:'` 前缀（含 `@<vendor>` 后缀）一并纳入；加会话级 `Map<String, IpGenOptions> ipGenOptions`（仿 `c_compile.dart:28` 的顶层 `sessionCompilerPaths`，非 `state` 字段）。
10. `lib/modules/isp_studio/widgets/node_canvas.dart` — 菜单项插在「查看黑盒子C代码」（:362）之后、`divider`（:364）之前（同属「代码/IP 查看」组），文案「生成Verilog IP」；分发加在 :390 前；仿 `viewGroupBlackBoxCode`（:416）加 `viewGroupIpCode`（先 `validateGroupIpExport` 校验不过弹错误，再打开 `ip_gen_dialog` 选厂商+参数、确定后 `state.openGroupIpTab(groupId, vendor:)`）。**注意与 C 代码的分歧**：C 代码走「原位级联 `pickGroupCTarget` 选 CPU」，IP 不走级联、走对话框——IP 除厂商外还有接口/PIX_BITS/MAX_WIDTH/GSTRATEGY/FOLD 等参数，对话框更合适。
11. `lib/modules/isp_studio/widgets/editor_tab_bar.dart` — :35-:37 图标、:53 `tabTitle` 加 `'gip:'` 分支（标题 `编组名 · IP · <厂商短名>`，仿黑盒分支解析 `@<vendor>` 后缀）。
12. `lib/modules/isp_studio/isp_studio_view.dart` — :144-:156 的 `IndexedStack` 分支加 `'gip:'` → `IpGeneratorPage`；同步扩展 :23/:29 的 `groupTabId`/`groupTabTarget` 或新写 `ipTabVendor` 解析 `@<vendor>` 后缀。

测试与文档

13. `test/isp_group_ip_export_test.dart` — 校验拒绝/放行用例；生成文件集结构断言（模块名/参数/generate 块/接口信号，含 `vidin` 端口列表）；`golden` 与 Dart 管线一致断言；**iverilog 实仿用例**（小图 16×16 + 边界尺寸矩阵，无 iverilog 自动 skip，仿黑盒测试 MSVC skip 模式）+ **vidin 结构仿真用例**（LVDS 排序规则/DVP 时序 → 像素流还原）；资源报告合理性。

14. `AGENTS.md` — 模块表 ISP Studio 条目追加 IP Generator 一段（注意与刚落地的 CPU 目标级联内容衔接：FPGA 厂商目标与 `GroupCTarget` CPU 目标并列）。
 15. `docs/IP_Generator.md` — 功能文档（接口时序、参数表、支持节点、验证方法、三家工具导入步骤）。
- `.iss` 无需改动（生成在内存/临时目录，无新运行时依赖目录）。

实施顺序（每步独立可验）

1. **校验 + 规划层**：ip_validate.dart + ip_gen_plan.dart + 单测（拒绝集/白名单/位深推导）。
2. **点操作与 LUT 节点发射器**（node_v_stream.dart 前半）+ group_ip_export 骨架（单节点编组可出 core.v）+ 单测。
3. **窗口节点发射器**（unroll 变体先行，fold 变体随后）+ 行缓冲/延迟均衡装配 + 单测。
4. **视频输入适配层**（vidin_stream.dart）：none/DVP 先行（纯并行）→ LVDS 解串 + 排序规则还原 → MIPI 厂商 RX 实例化最后；配套 vidin 结构仿真用例。
5. **三家输出封装 + tb + golden**（Dart 侧算 golden：输入 LCG 同 abMainSource 口径，输出走既有 Dart 管线核逐位口径；容差节点按 C lut 口径）+ package_ip.tcl + README。
6. **iverilog 探测 + 一键仿真**（iverilog_sim.dart）。
7. **UI 全链路**：对话框（含「视频输入接口」组）+ 页面 + 四处挂载点。
8. **收尾**：flutter analyze + 相关测试全绿 + [AGENTS.md](#) + docs/IP_Generator.md。

风险与对策

- **fold 变体工作量大**：窗口节点仅 4 个类型（morphology/dpc/demosaic/highlight recover），模板各两套，可控；若超期，fold 变体可降级为「参数存在但 v1 仅 unroll 生效，fold 在 README 标注待完成」——但不砍节点范围。
- **边界逐位一致**：demosaic/dpc 的裁剪语义（可变 count 除法）用精确小常量除法实现，测试含 1×1/1×7/2×2 等边界尺寸矩阵（仿黑盒测试口径）。
- **iverilog 不在场**：仿真用例 skip、UI 探测行提示安装，不影响导出功能本身。
- **LVDS 位对齐/排序规则还原易错**：排序规则用枚举 + 参数化 case 生成，行为仿真覆盖 bit-split/pixel-split × SDR/DDR 全组合；bitflip 训练留 v2（v1 假设 sensor 已对齐或提供训练模式）。
- **MIPI CSI-2 依赖厂商 RX IP**：生成器只做实例化 + 参数透传 + RAW 解包，不自行实现 D-PHY/包解析；无对应厂商 IP 时该接口目标标注「需外部 PHY」。
- **跨时钟域（sensor pixel clock → 系统时钟）**：vidin→core 之间用异步 FIFO 握手（ready 握手已具备），行缓冲天然吸收 CDC；时钟约束在 README 给出。